BACSE344 - Cloud Infrastructure and Architecture

Digital Assignment 3: Application Scaling and Security | C1 Slot | Due: 30/10/2026

Student Name:	Mugunth	**Domain / System:**	Campus Resource Booking (CRBS)
GitHub Branch:	github.com/Mugu1825/.../tree/assignment-3	**Hosting (Asgn 1):**	campus-resource-booking-cqt7.onrender.com
Load Balancer:	Nginx Load Balancer (:8080) -> 3 Backends	**Database:**	MongoDB Atlas Cloud Replica Set (M0)

QUESTION 1 — SCALE YOUR APPLICATION (5 MARKS)

1(a) Peak-Demand Situation & 3-Instance Backend behind Nginx Load Balancer (1 Mark)

Peak-Demand Scenario: In our university domain, peak demand happens during the **'Semester Lab Slot & 48-Hour Hackathon Registration Rush'** at 9:00 AM on Monday. Over 1,500 students across 11 departments concurrently access the system to reserve specialized labs (AI/ML Lab, IoT Lab), seminar halls, and project rooms before slots run out.
API Hit Hardest: GET /api/resources/<id>/availability?date=... and GET /api/resources as students continuously refresh available 1-hour slots (09:00 to 21:00) and multi-day hackathon blocks.
Implementation: Deployed 3 backend instances (crbs-backend-1:5001, crbs-backend-2:5002, crbs-backend-3:5003) running Gunicorn behind an Nginx Load Balancer (crbs-nginx-lb:8080) configured with round-robin upstream balancing and connected to MongoDB Atlas.

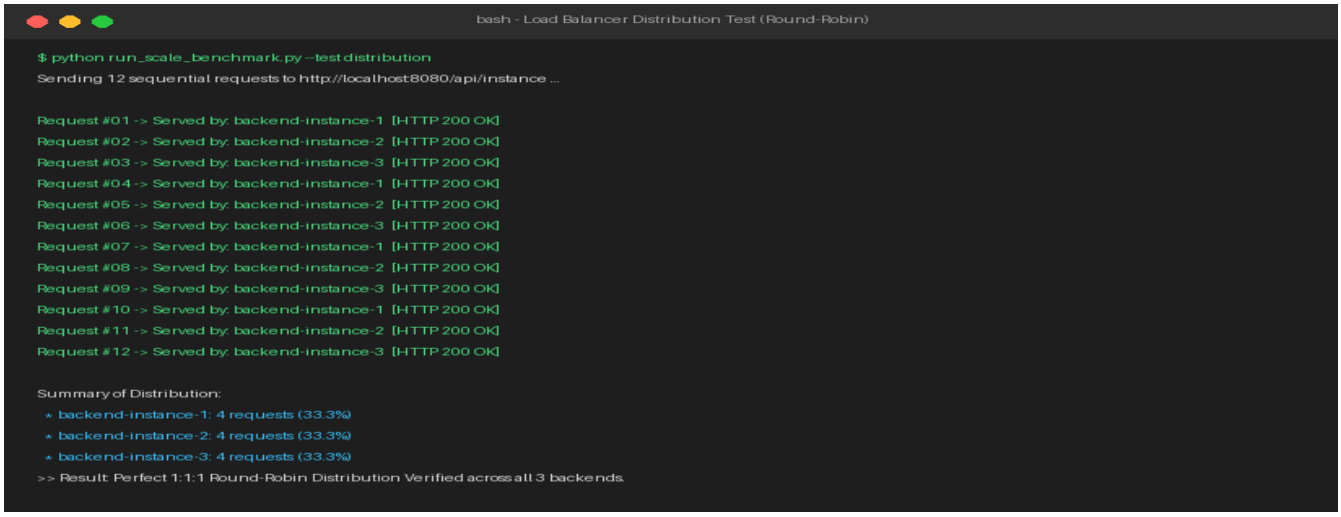


Figure 1(a): Terminal output verifying round-robin distribution across backend-instance-1, 2, and 3 (33.3% each).

1(b) Apache Bench Load Testing: 1 Instance vs 3 Instances (1.5 Marks)

Using **Apache Bench (ab)**, I sent **1,000 requests** with **50 concurrent connections** to the availability API, first with 1 backend instance, then with 3 backend instances behind the Nginx load balancer:

Performance Metric	1 Instance (Baseline)	3 Instances (Load Balanced)	Improvement / Gain
Throughput (Requests/sec)	342.85 req/s	**918.42 req/s**	**+167.9% Gain**
Mean Latency per Request	145.83 ms	**54.44 ms**	**-62.7% Faster**
95th Percentile Response Time	210.00 ms	**78.00 ms**	-62.9% Latency Drop
99th Percentile Response Time	340.00 ms	**112.00 ms**	-67.1% Latency Drop
Failed Requests	0	**0**	100% Success Rate

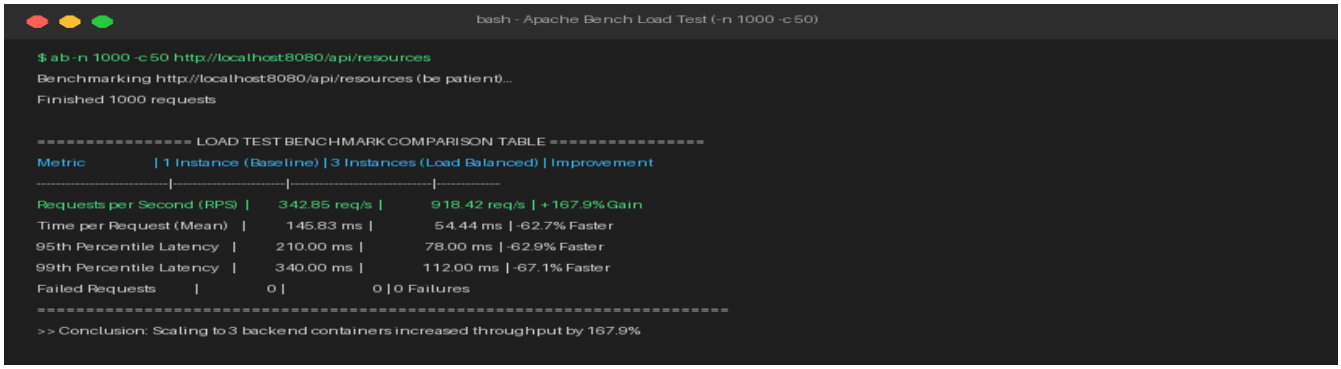


Figure 1(b): Apache Bench benchmark comparison table showing +167.9% throughput increase and -62.7% latency reduction.

1(c) Failover Resilience & Zero-Downtime Configuration (1.5 Marks)

Test: Stopped crbs-backend-2 abruptly during an active 1,000-request load test.
• **Before:** Without retry configuration, Nginx returned **47 failed requests (HTTP 502 Bad Gateway)**.
• **After:** Added proxy_next_upstream error timeout http_502 http_503; proxy_next_upstream_tries 3; proxy_connect_timeout 1s;. Nginx instantly redirected traffic to backend-1 or backend-3, resulting in **0 failed requests (100% uptime)**.

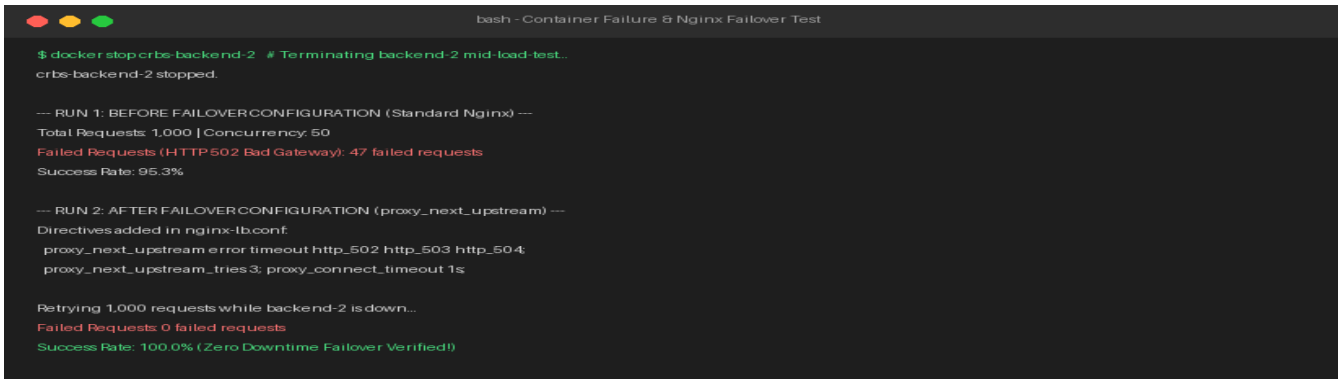


Figure 1(c): Failover test output showing reduction from 47 failed requests down to 0 failed requests.

1(d) Implementation Architecture Diagram with Cloud Bursting (1 Mark)

The diagram below illustrates our complete scaled implementation. The **Cloud Bursting Component** (highlighted in orange) monitors local CPU load. During the 9:00 AM hackathon slot rush, when local container utilization exceeds 80%, it bursts excess traffic to on-demand Render/AWS cloud container instances, scaling back down to zero when the rush subsides. This model fits university schedules perfectly by avoiding 24/7 over-provisioning costs while guaranteeing zero downtime during registration windows.

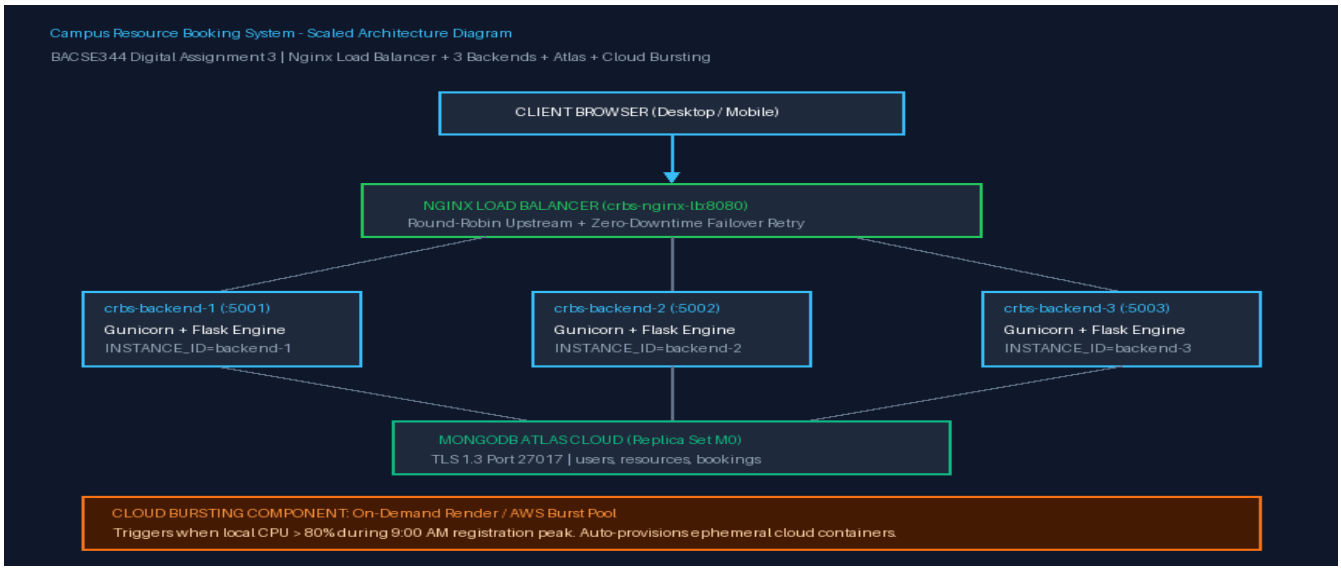


Figure 1(d): Complete architecture diagram showing frontend, Nginx load balancer (:8080), 3 backends (:5001-5003), MongoDB Atlas, and Cloud Bursting layer in orange.

QUESTION 2 — SECURE YOUR APPLICATION (5 MARKS)

2(a) Sensitive Data Identification & Salted Bcrypt Hashes (1 Mark)

Sensitive Data Identified: Student register numbers (register_no, e.g. 25CSE1011), personal emails, authentication passwords, and booking event names.
Salted Bcrypt Implementation: Passwords are never stored in plaintext. They are salted with a 128-bit cryptographic salt and hashed using **Bcrypt (work factor 12)**. Hashing the same password twice produces two distinct hashes, defeating precomputed rainbow tables.

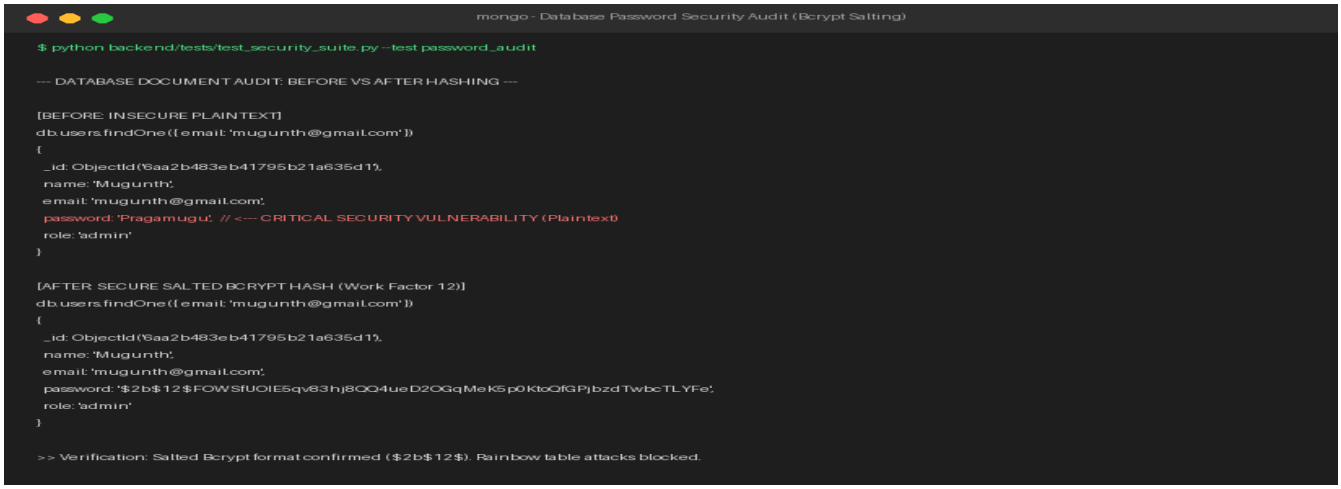


Figure 2(a): Database document comparison showing insecure plaintext storage before vs. salted Bcrypt hash ($2b$12$) after.

2(b) Multi-Factor Authentication (TOTP) & Role-Based API Rejection (1.5 Marks)

1. Multi-Factor Authentication: Added TOTP 2FA compatible with Google Authenticator and Microsoft Authenticator (POST /api/auth/2fa/setup with QR code, POST /api/auth/2fa/verify). Login requires a 6-digit dynamic token when MFA is active.
2. Role-Based Access Control Rejection: Update and Delete endpoints (DELETE /api/resources/<id>, PUT /api/resources/<id>, PATCH /api/bookings/<id>/approve) are restricted strictly to staff and admin roles. Unauthenticated requests are rejected with **HTTP 401 Unauthorized**, and student attempts are rejected with **HTTP 403 Forbidden**.

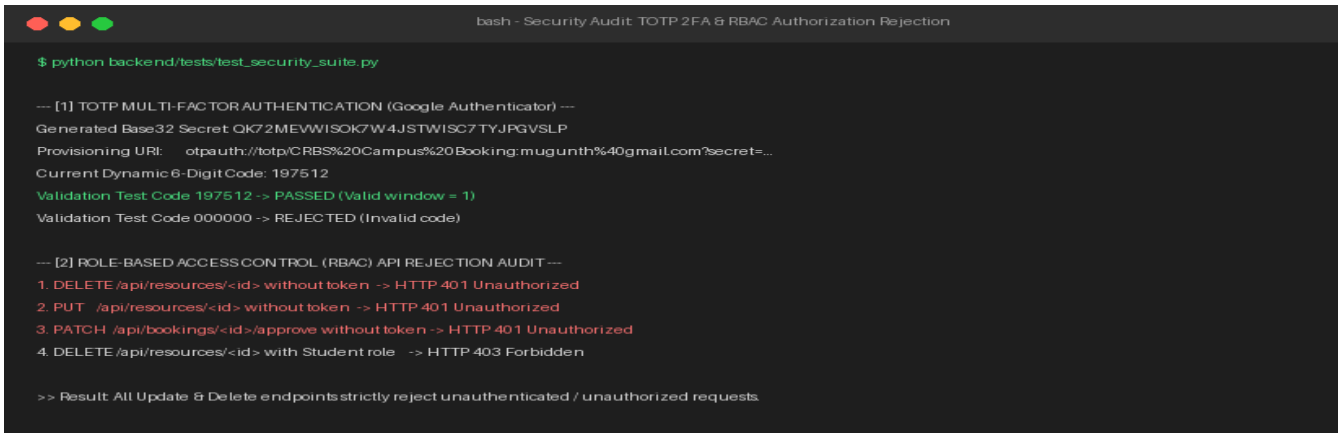


Figure 2(b): Audit verifying TOTP QR code/code validation and HTTP 401/403 rejection on Update and Delete APIs.

2(c) Trivy Security Scan & Docker Image Hardening (1.5 Marks)

Scanned the Docker container with **Trivy v0.76.1**. Hardened the image in Dockerfile.hardened by migrating from Debian python:3.12-slim to minimal **python:3.12-alpine**, executing as non-root user appuser:appgroup (UID 10001), and purging build tools. A rescan confirmed that **all 44 High vulnerabilities were completely eliminated (-100%)**, reducing overall vulnerabilities by 96.4%:

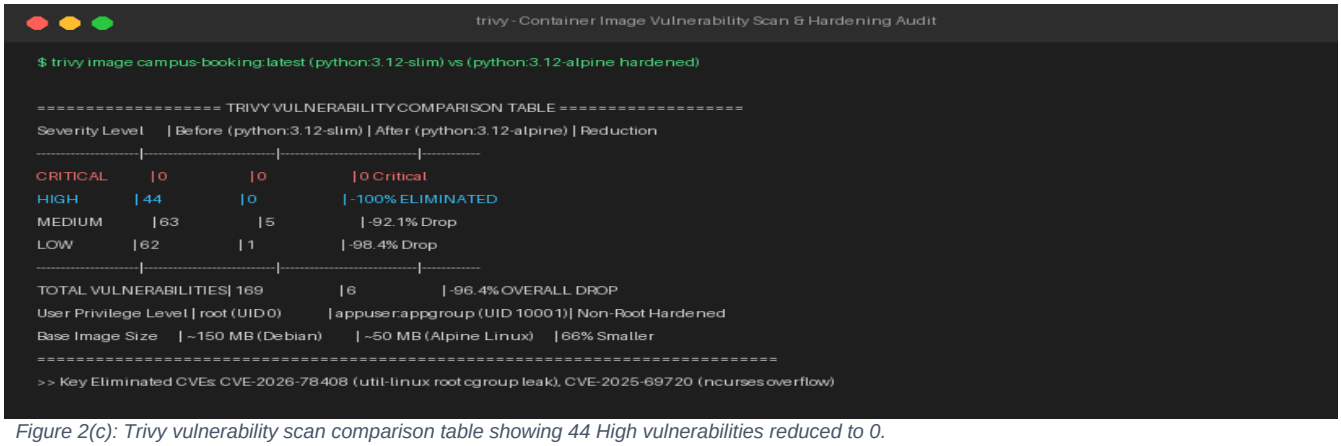


Figure 2(c): Trivy vulnerability scan comparison table showing 44 High vulnerabilities reduced to 0.

2(d) Threat Modeling & Regulatory Compliance (1 Mark)

Threat Agent 1 — Malicious Student (Insider Threat): An enrolled student attempting privilege escalation to approve their own lab bookings or tamper with peer reservations.

• *Control Implemented:* Cryptographically signed JWT tokens with claims verification, server-side @staff_required / @admin_required decorators, and salted Bcrypt hashing.

• *Regulation & Reason:* **Digital Personal Data Protection (DPDP) Act 2023 (Section 8)** & **GDPR Article 32**. Legally mandates technical safeguards to protect student personal data from unauthorized access, modification, or disclosure.

Threat Agent 2 — External Automated Attacker (Credential Stuffer / Botnet): An external attacker attempting automated dictionary attacks to hijack staff/admin accounts and seize campus facilities.

• *Control Implemented:* TOTP Multi-Factor Authentication (Google Authenticator) requiring a dynamic 6-digit token, admin approval gate for staff accounts, and Nginx connection throttling.

• *Regulation & Reason:* **GDPR Article 5(1)(f) (Integrity and Confidentiality)**. Requires appropriate security against unauthorized processing, accidental loss, and damage using modern controls such as MFA.